

Node.js RAG Reader Test Document

A simple 15-page PDF designed for testing PyPDFLoader, chunking, embeddings, retrieval, and question answering.

Unique document code: NODE-RAG-2026-ALPHA

Test person: Maya Verma

Test project: GreenCart API

Important retrieval fact: The demo server uses port 4321.

1. What is Node.js?

Node.js is a JavaScript runtime that executes JavaScript outside a web browser. It is commonly used for servers, command-line tools, automation scripts, and real-time applications.

Node.js uses the V8 JavaScript engine. Its event-driven model makes it suitable for applications that handle many input and output operations.

Key	Value
Runtime	Node.js
JavaScript engine	V8
Demo project	GreenCart API

RAG test marker: 1.-what-is-node.js?

2. Installing Node.js

After installing Node.js, verify the installation with `node --version` and `npm --version`. npm is the package manager normally installed with Node.js.

For this test document, assume the installed Node.js version is 22.11.0 and the npm version is 10.9.0.

```
node --version
npm --version
```

Key	Value
Test Node version	22.11.0
Test npm version	10.9.0

RAG test marker: 2.-installing-node.js

3. Creating a Project

Create a folder, open it in a terminal, and run `npm init -y`. This command creates a `package.json` file with default values.

The GreenCart API package name is `greencart-api` and its entry file is `src/server.js`.

```
mkdir greencart-api
cd greencart-api
npm init -y
```

Key	Value
Package name	greencart-api
Entry file	src/server.js

RAG test marker: 3.-creating-a-project

4. CommonJS Modules

CommonJS uses require to import modules and module.exports to export values. Many older Node.js projects use this format.

The sample tax module exports a function named calculateTax. The default test tax rate is 18 percent.

```
const calculateTax = (amount) => amount * 0.18;
module.exports = calculateTax;
```

Key	Value
Module format	CommonJS
Tax rate	18 percent

RAG test marker: 4.-commonjs-modules

5. ES Modules

ES modules use import and export syntax. A Node.js project can enable them by setting type to module in package.json or by using the .mjs extension.

The sample utility exports a function named formatPrice. It formats values using the INR currency code.

```
export function formatPrice(value) {  
  return new Intl.NumberFormat('en-IN', { style: 'currency', currency: 'INR' }).format(value);  
}
```

Key	Value
Syntax	import and export
Currency	INR

RAG test marker: 5.-es-modules

6. Asynchronous Programming

Node.js commonly uses promises and async/await for asynchronous operations. Await pauses only the current async function, not the whole server process.

The mock inventory request resolves after 250 milliseconds and returns 48 available items.

```
async function getInventory() {
  return { available: 48, delayMs: 250 };
}
```

Key	Value
Mock delay	250 milliseconds
Available items	48

RAG test marker: 6.-asynchronous-programming

7. File System Module

The built-in fs module reads and writes files. The promise-based API is available from node:fs/promises.

The sample application stores local backup data in data/backup.json. The backup runs every night at 11:30 PM.

```
import { readFile } from 'node:fs/promises';
const text = await readFile('data/backup.json', 'utf8');
```

Key	Value
Backup file	data/backup.json
Backup time	11:30 PM

RAG test marker: 7.-file-system-module

8. Building an HTTP Server

The built-in `node:http` module can create a web server without Express. The request object contains incoming request information, while the response object sends data back.

This document uses port 4321 for the demo server. A request to `/health` returns the word `healthy`.

```
import http from 'node:http';
http.createServer((req, res) => {
  res.end(req.url === '/health' ? 'healthy' : 'not found');
}).listen(4321);
```

Key	Value
Server port	4321
Health route	/health
Health response	healthy

RAG test marker: 8.-building-an-http-server

9. Express Basics

Express is a popular web framework for Node.js. It provides routing, middleware, request parsing, and response helpers.

The GreenCart API has a GET endpoint at `/api/products`. The endpoint returns exactly 12 demo products.

```
app.get('/api/products', (req, res) => {  
  res.json({ count: 12, items: [] });  
});
```

Key	Value
Endpoint	GET <code>/api/products</code>
Demo product count	12

RAG test marker: 9.-express-basics

10. Middleware

Middleware functions run during the request-response cycle. They can log requests, check authentication, parse JSON, or handle errors.

The sample authentication middleware expects a header named x-api-key. The test key is gc_test_9087.

```
const key = req.headers['x-api-key'];  
if (key !== 'gc_test_9087') return res.status(401).json({ error: 'Unauthorized' });
```

Key	Value
Header	x-api-key
Test API key	gc_test_9087

RAG test marker: 10.-middleware

11. Error Handling

Centralized error middleware normally accepts four parameters: error, request, response, and next. It should be registered after application routes.

The sample application uses error code `PRODUCT_NOT_FOUND` when a product does not exist.

```
app.use((err, req, res, next) => {  
  res.status(err.status || 500).json({ code: err.code || 'SERVER_ERROR' });  
});
```

Key	Value
Missing product code	PRODUCT_NOT_FOUND
Default status	500

RAG test marker: 11.-error-handling

12. Environment Variables

Environment variables keep configuration separate from source code. Typical examples include database URLs, API keys, ports, and runtime modes.

The GreenCart test database name is greencart_dev. The test environment value for NODE_ENV is development.

```
PORT=4321
DB_NAME=greencart_dev
NODE_ENV=development
```

Key	Value
Database name	greencart_dev
Environment	development

RAG test marker: 12.-environment-variables

13. Testing with Node Test Runner

Node.js includes a built-in test runner available through `node:test`. Assertions can be imported from `node:assert`.

The sample price test checks that a base price of 100 plus 18 percent tax equals 118. The test suite name is pricing rules.

```
import test from 'node:test';
import assert from 'node:assert/strict';
test('pricing rules', () => assert.equal(100 + 18, 118));
```

Key	Value
Suite name	pricing rules
Expected total	118

RAG test marker: 13.-testing-with-node-test-runner

14. RAG Retrieval Questions

This page contains questions that can be used to evaluate retrieval. A good RAG system should locate the supporting page instead of guessing.

Questions: What port does the demo server use? What is the test API key? How many products are returned? What time does backup run? Who is the test person?

Key	Value
Test person	Maya Verma
Unique code	NODE-RAG-2026-ALPHA
Project	GreenCart API

RAG test marker: 14.-rag-retrieval-questions

15. Final Verification Facts

The final verification phrase is: orange comet crosses seven rivers. This unusual phrase is intentionally included for exact-match retrieval testing.

The emergency inventory threshold is 9 units. The internal support email used only in this fictional document is support@greencart.example.

Key	Value
Verification phrase	orange comet crosses seven rivers
Inventory threshold	9 units
Support email	support@greencart.example

RAG test marker: 15.-final-verification-facts